```java
@SpringBootApplication
public class SpringBootBasicApplication {
   public static void main(String[] args) {
       //启动入口
       SpringApplication.run(SpringBootBasicApplication.class, args);
   }
}

ConfigurableApplicationContext SpringApplication.run(Class<?>[] primarySources, String[] args) {
   return new SpringApplication(primarySources).run(args);
}
```

```java
… → ConfigurableApplicationContext SpringApplication.run(String... args) {
  DefaultBootstrapContext bootstrapContext = createBootstrapContext();
  ConfigurableApplicationContext context = null;
  configureHeadlessProperty();
  SpringApplicationRunListeners listeners = getRunListeners(args);
  listeners.starting(bootstrapContext, this.mainApplicationClass);
  try {
    ApplicationArguments applicationArguments = new DefaultApplicationArguments(args);
    ConfigurableEnvironment environment = prepareEnvironment(listeners, bootstrapContext, applicationArguments);
    configureIgnoreBeanInfo(environment);
    Banner printedBanner = printBanner(environment);
    context = createApplicationContext(); //1.create ApplicationContext context
    context.setApplicationStartup(this.applicationStartup);
    //2.注册SpringBootBasicApplication的beanDefinition
    prepareContext(bootstrapContext, context, environment, listeners, applicationArguments, printedBanner);
    //3.refresh the application context, load all singleton beans
    refreshContext(context);
    afterRefresh(context, applicationArguments);
    Duration timeTakenToStartup = Duration.ofNanos(System.nanoTime() - startTime);
    if (this.logStartupInfo) new StartupInfoLogger(this.mainApplicationClass).logStarted(getApplicationLog(), timeTakenToStartup);
    listeners.started(context, timeTakenToStartup);
    callRunners(context, applicationArguments);  //4.trigger any CommandLineRunner beans
  }
  try {
    Duration timeTakenToReady = Duration.ofNanos(System.nanoTime() - startTime);
    listeners.ready(context, timeTakenToReady);
  }
  return context;
}
```

```java
AbstractApplicationContext.refresh() {
  synchronized (this.startupShutdownMonitor) {
    StartupStep contextRefresh = this.applicationStartup.start("spring.context.refresh");
    prepareRefresh();
    ConfigurableListableBeanFactory beanFactory = obtainFreshBeanFactory();
    prepareBeanFactory(beanFactory);
    try {
      postProcessBeanFactory(beanFactory);
      StartupStep beanPostProcess = this.applicationStartup.start("spring.context.beans.post-process");
      invokeBeanFactoryPostProcessors(beanFactory); //自动装配
      registerBeanPostProcessors(beanFactory);
      beanPostProcess.end();
      initMessageSource();
      initApplicationEventMulticaster();
      onRefresh(); //创建WebServer
      registerListeners();
      finishBeanFactoryInitialization(beanFactory); //实例化beans
      finishRefresh(); //启动WebServer
    }
  }
}
```

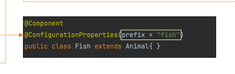
创建Fish的实例:
1. createBeanInstance: instantiate the bean
2. applyMergedBeanDefinitionPostProcessors: allow post-processors to modify the merged bean definition
3. populateBean: 注入依赖
4. initializeBean
   > applyBeanPostProcessorsBeforeInitialization()
     > ConfigurationPropertiesBindingPostProcessor.postProcessBeforeInitialization()
5. registerDisposableBeanIfNecessary

> ConfigurationPropertiesBindingPostProcessor在每个bean注入依赖后，检查其声明类是否带有@ConfigurationProperties注解
  > 如果没有，直接返回
  > 如果有 > 获取注解中的prefix
        > 遍历bean的所有fields，以prefix.field为key，查找缓存到内存中的配置项
          如果找到了，且field有setter方法，反射执行setter方法将value设置到field中（此时field如果有值，会被覆盖）

```java
Object ConfigurationPropertiesBindingPostProcessor.postProcessBeforeInitialization(Object bean, String beanName) {
  bind(ConfigurationPropertiesBean.get(this.applicationContext, bean, beanName));
  return bean;
}
```

```java
ConfigurationPropertiesBean ConfigurationPropertiesBean.create(String name, Object instance, Class<?> type, Method factory) {
  //获取注解信息
  ConfigurationProperties annotation = (ConfigurationProperties)findAnnotation(instance, type, factory, ConfigurationProperties.class);
  if (annotation == null)  return null;
  else {
    Validated validated = (Validated)findAnnotation(instance, type, factory, Validated.class);
    Annotation[] annotations = validated ≠ null ? new Annotation[]{annotation, validated} : new Annotation[]{annotation};
    ResolvableType bindType = factory ≠ null ? ResolvableType.forMethodReturnType(factory) : ResolvableType.forClass(type); //com.javadevmap.basic.Fish
    Bindable<Object> bindTarget = Bindable.of(bindType).withAnnotations(annotations);
    if (instance ≠ null)  bindTarget = bindTarget.withExistingValue(instance);
    return new ConfigurationPropertiesBean(name, instance, annotation, bindTarget);
  }
}
```

```java
BindResult<?> ConfigurationPropertiesBinder.bind(ConfigurationPropertiesBean propertiesBean) {
  Bindable<?> target = propertiesBean.asBindTarget();
  ConfigurationProperties annotation = propertiesBean.getAnnotation();
  BindHandler bindHandler = this.getBindHandler(target, annotation);
  //ConfigurationProperties注解的prefix是获取配置值的关键
  return this.getBinder().bind(annotation.prefix(), target, bindHandler);
}
```



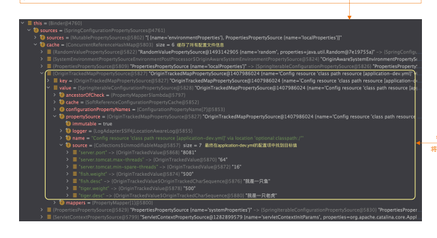
```java
@Component
@ConfigurationProperties(prefix = "fish")
public class Fish extends Animal{ }
```

```java
… → <T> Object Binder.bindDataObject(ConfigurationPropertyName name, Bindable<?> target, BindHandler handler, Binder.Context context, boolean allowRecursiveBinding) {
  Class<?> type = target.getType().resolve(Object.class); //com.javadevmap.basic.Fish
  DataObjectPropertyBinder propertyBinder = (propertyName, propertyTarget) → bind(name.append(propertyName), propertyTarget, handler, context, false, false);
  return context.withDataObject(type, () → {
      for (DataObjectBinder dataObjectBinder : this.dataObjectBinders) {
          Object instance = dataObjectBinder.bind(name, target, context, propertyBinder);
          if (instance ≠ null) return instance;
      }
      return null;
  });
}
```

```java
… → <T> boolean JavaBeanBinder.bind(DataObjectPropertyBinder propertyBinder, Bean<T> bean, BeanSupplier<T> beanSupplier, Context context) {
  boolean bound = false;
  for (BeanProperty beanProperty : bean.getProperties().values()) { //获取bean的所有成员变量
     bound |= bind(beanSupplier, propertyBinder, beanProperty); //赋值
     context.clearConfigurationProperty();
  }
  return bound;
}

<T> boolean bind(BeanSupplier<T> beanSupplier, DataObjectPropertyBinder propertyBinder, BeanProperty property) {
  String propertyName = property.getName(); //desc
  ResolvableType type = property.getType(); //java.lang.String
  Supplier<Object> value = property.getValue(beanSupplier);
  Annotation[] annotations = property.getAnnotations();
  //1.查找配置的属性值
  Object bound = propertyBinder.bindProperty(propertyName, Bindable.of(type).withSuppliedValue(value).withAnnotations(annotations));
  if (bound == null) return false;
  if (property.isSettable()) property.setValue(beanSupplier, bound); //2.如果bean有setter方法，反射调用setter()给fish.desc赋值
  else if (value == null || !bound.equals(value.get())) throw new IllegalStateException("No setter found for property: " + property.getName());
  return true;
}
```

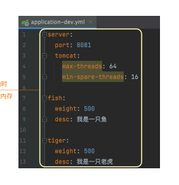
```java
… → <T> Object Binder.bindObject(ConfigurationPropertyName name, Bindable<T> target, BindHandler handler,
                                 Binder.Context context, boolean allowRecursiveBinding) {
  ConfigurationProperty property = this.findProperty(name, target, context);
  this.bindProperty(target, context, property);
}

<T> Object bindProperty(Bindable<T> target, Binder.Context context, ConfigurationProperty property) {
    context.setConfigurationProperty(property);
    Object result = property.getValue(); //我是一只鱼
    result = this.placeholdersResolver.resolvePlaceholders(result);
    result = context.getConverter().convert(result, target);
    return result; //我是一只鱼
}
```

```java
<T> ConfigurationProperty findProperty(ConfigurationPropertyName name, Bindable<T> target, Context context) {
  for (ConfigurationPropertySource source : context.getSources()) {
     ConfigurationProperty property = source.getConfigurationProperty(name); //name=fish.desc
     if (property ≠ null) return property;
  }
  return null;
}
```



```
this = {Binder@4760}
  sources = {SpringConfigurationPropertySources@4761}
    sources = {MutablePropertySources@5802} "[{name='environmentProperties'}, PropertiesPropertySource {name='localProperties'}]"
    cache = {ConcurrentReferenceHashMap@5803}  size = 6 缓存了所有配置文件信息
      {RandomValuePropertySource@5822} "RandomValuePropertySource@1493142905 {name='random', properties=java.util.Random@7e19755a}" -> {SpringConfigu
      {SystemEnvironmentPropertySourceEnvironmentPostProcessor$OriginAwareSystemEnvironmentPropertySource@5824} "OriginAwareSystemEnvironmentPropertySo
      {PropertiesPropertySource@5809} "PropertiesPropertySource {name='localProperties'}" -> {SpringIterableConfigurationPropertySource@5826} "PropertiesProperty
      {OriginTrackedMapPropertySource@5827} "OriginTrackedMapPropertySource@1407986024 {name='Config resource 'class path resource [application-dev.yml]' v
        key = {OriginTrackedMapPropertySource@5827} "OriginTrackedMapPropertySource@1407986024 {name='Config resource 'class path resource [application-d
        value = {SpringIterableConfigurationPropertySource@5828} "OriginTrackedMapPropertySource@1407986024 {name='Config resource 'class path resource [ap
          ancestorOfCheck = {PropertyMapper$lambda@5797}
          cache = {SoftReferenceConfigurationPropertyCache@5852}
          configurationPropertyNames = {ConfigurationPropertyName[7]@5853}
          propertySource = {OriginTrackedMapPropertySource@5827} "OriginTrackedMapPropertySource@1407986024 {name='Config resource 'class path resource
            immutable = true
            logger = {LogAdapter$Slf4jLocationAwareLog@5855}
            name = "Config resource 'class path resource [application-dev.yml]' via location 'optional:classpath:/'"
            source = {Collections$UnmodifiableMap@5857}  size = 7  最终在application-dev.yml的配置项中找到目标值
              "server.port" -> {OriginTrackedValue@5868} "8081"
              "server.tomcat.max-threads" -> {OriginTrackedValue@5870} "64"
              "server.tomcat.min-spare-threads" -> {OriginTrackedValue@5872} "16"
              "fish.weight" -> {OriginTrackedValue@5874} "500"
              "fish.desc" -> {OriginTrackedValue$OriginTrackedCharSequence@5876} "我是一只鱼"
              "tiger.weight" -> {OriginTrackedValue@5878} "500"
              "tiger.desc" -> {OriginTrackedValue$OriginTrackedCharSequence@5880} "我是一只老虎"
          mappers = {PropertyMapper[1]@5800}
      {PropertiesPropertySource@5829} "PropertiesPropertySource {name='systemProperties'}" -> {SpringIterableConfigurationPropertySource@5830} "PropertiesProper
      {ServletContextPropertySource@5799} "ServletContextPropertySource@1282899579 {name='servletContextInitParams', properties=org.apache.catalina.core.Appl
```

springboot启动时
将yml的配置读入内存



```yaml
server:
  port: 8081
  tomcat:
    max-threads: 64
    min-spare-threads: 16

fish:
  weight: 500
  desc: 我是一只鱼

tiger:
  weight: 500
  desc: 我是一只老虎
```